Person A: Voice and API layer

Backend assignment – AI Service Navigator (voice assistant for finding Caritas support services, DE / EN)

Role	Works with	Depends on	Stack
Backend, voice + API	Person B (brain), frontend team	Person B's navigate() – stub it until ready	FastAPI, faster-whisper, Piper (all open source, CPU)

1. Your mission

You own everything between the browser and the "brain": receive audio, turn it into text, hand the text to Person B's navigation function, turn the reply into speech, and return one clean JSON response. You also own the session memory, error handling, speed, and the contract with the frontend team. Person B owns what the assistant *says*; you own how it is *heard and delivered*.

Every endpoint and field below must match the Frontend Handoff Specification, which the frontend team is building against.

2. Deliverables

#	Deliverable	Done when
1	FastAPI app with `POST /talk`, `GET /audio/{id}.wav`, `GET /health`, CORS enabled	Frontend can call all three from `localhost` without CORS errors.
2	Speech-to-text with faster-whisper, auto language detection (DE/EN) and manual override	German and English clips are transcribed correctly; override beats detection.
3	Text-to-speech with Piper, one German and one English voice, chosen per reply language	Reply audio plays in the right language and is clear.
4	In-memory session store with history per `session_id`	Follow-up turns receive earlier turns; old sessions expire.
5	Error codes and friendly JSON errors	Each code in section 7 can be triggered and returns the specified status.
6	Startup model loading, per-stage latency logging	First request is not slow; timings appear in logs.
7	Mock mode (`MOCK=1`)	Server runs and answers without any model installed.
8	Short README: install, run, env vars, how to test with curl	A new teammate can run it in 10 minutes.

3. The contract with Person B

Agree on this in the first 15 minutes and keep it in `contracts.py`. Do not change signatures without telling Person B. You implement the first two functions; Person B implements `navigate`.

```
# contracts.py
# Person A implements
transcribe(audio_bytes: bytes, language_hint: str | None) -> {"text": str, "language": "de" | "en"}
synthesize(text: str, language: str) -> str          # path to a .wav file

# Person B implements (you call it; write a stub until B delivers)
navigate(session_id: str, history: list[dict], user_text: str, language: str) -> {
    "reply_text": str,
    "needs_more_info": bool,
    "service": None | {"id", "name", "why", "address", "phone",
                       "email", "website", "opening_hours", "languages"}
}
```

Stub for the first two hours, so you never wait on Person B:

```
def navigate(session_id, history, user_text, language):
    return {"reply_text": "Pflegen Sie Ihre Mutter zu Hause?" if language == "de"
            else "Do you care for your mother at home?",
            "needs_more_info": True, "service": None}
```

4. Endpoint specification

POST /talk

Content type `multipart/form-data`. Exactly one of `audio` or `text` must be present.

Field	Type	Rules
audio	file, webm/opus or wav	Max 30 s. Reject smaller than about 1 KB or shorter than 0.3 s as `empty_audio`.
text	string	Typing fallback. Skips speech-to-text.
language	“auto” | “de” | “en”	Default “auto”. If “de” or “en”, use it and skip detection.
session_id	string (UUID)	Created by the frontend. Create a new session if unknown.

Response 200

```
{
  "session_id": "b1f3...9c",
  "transcript": "Ich weiß nicht mehr weiter, meine Mutter ist pflegebedürftig",
  "detected_language": "de",
  "reply_text": "Pflegen Sie Ihre Mutter zu Hause?",
  "reply_audio_url": "/audio/7a2c.wav",
  "needs_more_info": true,
  "service": null,
  "timings": {"stt_ms": 1800, "nav_ms": 2400, "tts_ms": 600, "total_ms": 4800}
}
```

The `timings` field is an extra for debugging; the frontend ignores it. When a service is recommended, `needs_more_info` is false and `service` is the object from Person B, passed through unchanged.

Other endpoints

Endpoint	Behavior
GET /audio/{id}.wav	Serves the generated reply file with `audio/wav`. Return 404 if missing. Delete files older than 1 hour.
GET /health	Returns `{"ok": true}` once models are loaded; `{"ok": false}` while loading. Include the model names for debugging.

5. Speech-to-text (faster-whisper)

- Model: `small`, `device="cpu"`, `compute_type="int8"`. Fall back to `base` if one turn takes more than about 3 s for 10 s of speech. Use a GPU model only if one is available.
- Load the model **once at startup** (FastAPI lifespan), never per request.
- Call with `vad_filter=True` so silence is trimmed and empty clips return no text.
- Language: with `language=None`, Whisper detects any language. Restrict it to German and English by comparing the probabilities for de and en from `info.all_language_probs` and taking the higher one. Short utterances often misdetect, so honor the manual override from the frontend toggle.
- Decoding: browser recordings arrive as webm/opus. faster-whisper uses PyAV, which normally decodes this directly. Test it on day one; if it fails, install ffmpeg and convert to 16 kHz mono wav.
- Text input: if `language` is “auto”, detect it with a small library such as `langdetect` and default to German on failure.

6. Text-to-speech (Piper), sessions and concurrency

Text-to-speech

- Install `piper-tts`. Download one German voice (for example `de_DE-thorsten-medium`) and one English voice (for example `en_US-lessac-medium`) into a `models/` folder.
- Voice licenses differ. Check each voice's model card and note the license in the README.
- Select the voice by the language of the **reply**, which follows the user's language.
- Write each reply to `tmp/audio/{uuid}.wav` and return its URL. Strip markdown or emojis before synthesis.
- Test pronunciation of service names, addresses and phone numbers; expand abbreviations such as "Mo–Fr" if they sound wrong.

Sessions

- Dictionary keyed by `session_id`: `{"history": [{"role": "user"|"assistant", "text": str}], "last_seen": ts}`.
- Append the user turn and the assistant turn after each successful request; pass `history` to `navigate`.
- Expire sessions after 60 minutes idle. Cap history at the last 10 turns.
- Use a lock around session writes. No database is needed for the prototype.

Concurrency

- Run Whisper, the LLM call and Piper in a thread pool so they do not block the event loop.
- Use one inference lock or semaphore. If more than 3 requests are waiting, return `server_busy`.

7. Error handling

All errors return `{"error": "<code>", "message": "<short English text>"}`. The frontend maps each code to a friendly DE/EN message.

Code	HTTP	When
empty_audio	400	No audio part, file too small, or shorter than 0.3 s.
no_speech	422	Transcription is empty or voice-activity detection finds no speech.
bad_request	400	Neither or both of audio and text sent; invalid language value.
server_busy	503	Too many requests waiting.
internal	500	Anything else. Log the traceback; never return it to the client.

If `navigate` raises or returns an invalid shape, respond with `internal` and log the input so Person B can reproduce it.

8. Latency and performance

- Log per-stage timings on every request: `stt_ms`, `nav_ms`, `tts_ms`, `total_ms`.
- Expected on CPU: 4–8 s per turn. Target under 5 s for a 10-second utterance. Report the real numbers to the team by hour 2 so the frontend can tune its thinking state.
- Warm up all models with a dummy call at startup.
- Biggest wins if too slow: a smaller Whisper model, a shorter reply (ask Person B to cap reply length), and a quantized or smaller LLM.

9. Mock mode

`MOCK=1` starts the server without loading any model. `/talk` returns canned responses: the first call returns a follow-up question, the second returns a full `service` object, in German or English depending on the `language` field. A short silent or beep wav can serve as `reply_audio_url`. The frontend team can integrate against this on day one, even on a laptop without the models.

10. Suggested project layout

```
backend/
  main.py            # FastAPI app, routes, CORS, lifespan (model loading)
  contracts.py       # shared function signatures (agreed with Person B)
  voice/
    stt.py           # transcribe()
    tts.py           # synthesize()
    lang.py          # language selection helpers
  sessions.py        # in-memory session store with expiry
  errors.py          # error codes and exception handlers
  mock.py            # canned responses for MOCK=1
  brain/             # Person B's folder - do not edit
  models/            # Piper voices (not committed)
  tmp/audio/         # generated wav files
  requirements.txt   # fastapi, uvicorn, python-multipart, faster-whisper,
                     # piper-tts, langdetect
```

Run: `uvicorn main:app --host 0.0.0.0 --port 8000`. Test: `curl -F "text=Ich brauche Hilfe" -F language=auto -F session_id=test1 localhost:8000/talk`.

11. Timeline

When	What
0:00–0:15	Agree on `contracts.py` and folder layout with Person B. Create the repo and your branch.
0:15–0:45	FastAPI skeleton, `/health`, CORS, mock mode. Share the mock URL with the frontend team.
0:45–1:30	Whisper working: language detection, override, empty-audio handling.
1:30–2:00	Piper working, both voices, `/audio/{id}.wav`. Full audio loop with the stub `navigate`.
2:00	**Integration checkpoint** with Person B: swap in the real `navigate`. Fix whatever breaks.
2:00–4:00	Sessions, error codes, latency tuning, end-to-end testing with the frontend in German and English.
4:00–5:00	README, cleanup of temp files, demo run-through. Help Person B with data or prompt tests if you finish early.

12. Acceptance checklist

#	Check
1	A German spoken request returns a transcript, a spoken German reply and the correct JSON shape.
2	The same in English; the manual language override works in both directions.
3	A second request with the same `session_id` passes earlier turns to `navigate`.
4	Every error code in section 7 can be triggered and returns the specified HTTP status.
5	First request after startup is not noticeably slower than later ones.
6	The frontend runs against the backend with no CORS errors and no field-name mismatches.
7	With `MOCK=1` the server runs on a machine with no models installed.
8	Timings are logged, and the real per-turn latency has been reported to the team.